

Solución Greedy

Idea General

La estrategia o el algoritmo Greedy consiste en tomar en cada paso la mejor decisión local posible esperando obtener una buena solución global.

En este problema, la decisión local óptima será:

- Seleccionar la tarea disponible con mayor prioridad y asignarla al servidor menos cargado.

Decisión Local Óptima: Seleccionar la tarea disponible más importante

Entre las tareas que: aún no han sido ejecutadas, y cumplen sus dependencias, se elige lo siguiente:

- la tarea con menor valor de prioridad.
- El algoritmo busca el servidor con menor carga acumulada, ya que esto nos ayudará a balancear el trabajo entre los servidores

Complejidad del Algoritmo Greedy

Complejidad temporal: en cada iteración hacemos lo siguiente: se buscan tareas disponibles, se ordenan por prioridad y se seleccionan los servidores menos cargados, esto nos da como resultado:

$O(n^2)$ porque el algoritmo recorre las tareas múltiples veces para validar dependencias, ordenar prioridades y asignar servidores, pero no explora todas las combinaciones posibles.

Complejidad Espacial: $O(n)$ porque almacena las cargas, las tareas ejecutadas y las asignaciones correspondientes.

¿La solución greedy es óptima?

No siempre, ya que esta estrategia toma decisiones locales óptimas pero no evalúa las consecuencias futuras ni compara múltiples caminos posibles.

Contraejemplo

```
tareas_grande = [  
    ("T1", 3, 1, []),  
    ("T2", 5, 2, ["T1"]),  
    ("T3", 2, 1, ["T1"]),
```

```

    ("T4", 4, 3, ["T2"]),
    ("T5", 6, 2, ["T2", "T3"]),
    ("T6", 3, 1, ["T3"]),
    ("T7", 7, 2, ["T4", "T5"]),
    ("T8", 2, 3, ["T6"]),
    ("T9", 5, 1, ["T6"]),
    ("T10", 4, 2, ["T7", "T8"]),
    ("T11", 6, 3, ["T9"]),
    ("T12", 3, 2, ["T10", "T11"])
]

```

Con estas líneas de tareas, nos damos cuenta que nuestro algoritmo greedy es demasiado veloz, pero a la hora de escoger localmente decisiones no siempre es la buena, ya que en este caso nos da como resultado un tiempo total de 26 en desarrollar estas tareas, pero con algoritmos como el de backtracking nos da un tiempo total de 25 en desarrollar las tareas, aunque con backtracking el algoritmo tarda 1.486267 segundos, y con greedy 0.003035 segundos. No es la mejor decisión pero sí de las más eficientes.

Solución con Backtracking

Idea General

Backtracking es una mejora de la fuerza bruta.

La diferencia es que el backtracking descarta las ramas inválidas o innecesarias antes de explorarlas completamente.

¿Qué poda el algoritmo? el algoritmo corta las ramas cuando se incumplen dependencias, también cuando la carga actual ya es peor que la mejor solución encontrada.

```
max(cargas) >= mejor_tiempo
```

Ya que si esto se cumple en nuestro código no vale la pena seguir explorando porque ya es peor que la mejor solución que se lleva al momento.

Complejidad del Backtracking

En el peor caso será $O(m^n)$ por que en un peor escenario aun puede explorar demasiadas combinaciones.

Caso promedio: Será mucho mejor que la fuerza bruta gracias a las podas, y esto mejora gracias a que evita explorar soluciones inválidas y corta las ramas peores.

Este mismo puede encontrar soluciones óptimas con mucho menos trabajo que el fuerza bruta.

Comparativa de tiempo con estas tareas.

```
tareas_grande = [  
    ("T1", 3, 1, []),  
    ("T2", 5, 2, ["T1"]),  
    ("T3", 2, 1, ["T1"]),  
    ("T4", 4, 3, ["T2"]),  
    ("T5", 6, 2, ["T2", "T3"]),  
    ("T6", 3, 1, ["T3"]),  
    ("T7", 7, 2, ["T4", "T5"]),  
    ("T8", 2, 3, ["T6"]),  
    ("T9", 5, 1, ["T6"]),  
    ("T10", 4, 2, ["T7", "T8"]),  
    ("T11", 6, 3, ["T9"]),  
    ("T12", 3, 2, ["T10", "T11"])
```

Tecnica	Tiempo
Fuerza Bruta	Tiempo de ejecución: 0.081512 segundos
Recursividad	Tiempo de ejecución: 5.899441 segundos
Greedy	Tiempo de ejecución: 0.003035 segundos
Backtraking	Tiempo de ejecución: 1.486267 segundos

Comparativa Parcial

Técnica	Complejidad Temporal	¿Óptima?	Velocidad	Dificultad
Fuerza Bruta	$O(m^n)$	Sí, siempre y cuando el tiempo no sea una restricción	Muy lenta	Fácil
Recursividad	$O(m^n)$	Sí, siempre y cuando el tiempo no sea una restricción	Muy lenta	Media
Greedy	$O(n^2)$	Si, siempre y cuando el tiempo de este sea premiado y no haya una restricción del tiempo al resolver las tareas	Muy rápida	Fácil
Backtracking	$O(m^n)$	Si, siempre y cuando el tiempo no sea una restricción	Mucho más rápida que fuerza bruta	Alta

Discusión de Trade-offs

Fuerza bruta:

Ventajas: Garantiza una solución óptima y fácil de entender.

Desventajas: Extremadamente lenta y poco escalable.

Recursividad:

Ventajas: Representa mejor el árbol de las decisiones y un código más “bonito”.

Desventajas: Consume mucha más memoria y riesgo de desbordamiento de pila.

Greedy:

Ventajas: Muy rápido, fácil de implementar y escalable.

Desventajas: “Como se mencionó anteriormente no encontramos un contraejemplo que muestre fallas a la hora de las decisiones y del tiempo”. Puede que este no garantiza la mejor solución ni el mejor tiempo.

Backtracking:

Ventajas: Encuentra soluciones óptimas y reduce muchísimo el espacio de búsqueda.

Desventajas: Muchísimo más complejo de implementar y aun puede ser costoso en casos muy grandes.